

Tensors, Worked Out by Hand

Shape, dtype, device — and the memory underneath

A complete numerical walk-through of how one small tensor is laid out in memory: how its *shape* becomes *strides*, how a multi-dimensional index becomes a single flat offset, and how *dtype* and *device* decide the bytes. Nothing is hand-waved — every number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It picks one tiny tensor — a $2 \times 3 \times 4$ block of coffee-shop sales — fixes its contents to the integers $0, 1, \dots, 23$ so the layout is impossible to misread, and then computes every property the course tells you to “always check”: **shape**, **dtype**, **device**, and the addressing math that connects them to physical memory.

How to read this. Each idea is explained on its own and gets a *Mini-example* box isolating that operation on the cleanest possible numbers, alongside the running sales-tensor example. Read this to understand what a tensor *is* underneath `torch.Size([2,3,4])`; the later topics (broadcasting, reshape/permute, contiguity) build directly on the stride picture established here.

Concepts covered, in order: what a tensor is (rank/axes) · shape, `ndim`, `numel` · row-major (C-contiguous) memory · the stride vector · the index→offset addressing equation · **dtype**, item size and total memory footprint · casting and rounding (`float16`) · **device** (CPU vs GPU) · a first look at why **reshape** is free and **transpose** breaks contiguity.

1 The setup: a tensor small enough to compute by hand

Real tensors hold millions of elements. You cannot follow that on paper, so we shrink to the smallest block that still exercises every mechanism — more than one axis, and more than one element per axis — and we fill it with $0 \dots 23$ so each value *is* its own position in memory.

Quantity	Symbol	Value here
Rank (number of axes)	n	3
Shape (size per axis)	(d_0, d_1, d_2)	(2, 3, 4)
Number of elements	<code>numel</code>	$2 \cdot 3 \cdot 4 = 24$
Element type	<code>dtype</code>	<code>float32</code> (4 bytes each)
Location	<code>device</code>	<code>cpu</code>
Memory order	—	row-major (C-contiguous)

The example. A chain tracks unit sales for 2 branches $\times$ 3 days $\times$ 4 items. Axis 0 is the branch, axis 1 the day, axis 2 the item. We fix the contents to `arange(24)`:

$$T[0] = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{bmatrix}, \quad T[1] = \begin{bmatrix} 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 \\ 20 & 21 & 22 & 23 \end{bmatrix}.$$

$T[0]$ is branch 0's 3×4 table of (day, item) sales; $T[1]$ is branch 1's. The value at position (i, j, k) is, by construction, exactly $12i + 4j + k$ — keep that in your pocket; Section 4 derives the same expression from the shape alone.

Intuition

A tensor is not a grid floating in space — it is one flat, contiguous run of numbers in memory plus a *recipe* for reading that run as an n -dimensional box. The recipe is the pair (shape, strides). Almost everything that feels mysterious later (why `view` is free, why `transpose` needs no copy, why some ops demand `.contiguous()`) is just this recipe being rewritten while the underlying run of numbers stays put.

2 Step 1 — Shape, `ndim` and `numel`

The **shape** is the tuple of axis lengths, $(d_0, d_1, d_2) = (2, 3, 4)$. Two scalars fall straight out of it:

$$\text{ndim} = n = 3 \quad (\text{how many numbers are in the shape tuple}),$$

$$\text{numel} = \prod_{a=0}^{n-1} d_a = 2 \times 3 \times 4 = 24 \quad (\text{how many elements the box holds}).$$

`ndim` answers "how many axes?"; `numel` answers "how many values?". They are different questions: a 1000-element vector has `ndim` = 1 but `numel` = 1000.

Mini-example — this operation in isolation

Rank is the count of axes, *not* of elements. Walking up the ladder with one element per new axis:

$$\underbrace{5}_{0\text{-D, ndim}=0}, \quad \underbrace{[5]}_{1\text{-D, (1)}}, \quad \underbrace{[[5]]}_{2\text{-D, (1,1)}}, \quad \underbrace{[[[5]]]}_{3\text{-D, (1,1,1)}}.$$

All four store a single number, so `numel` = 1 throughout, yet `ndim` climbs 0, 1, 2, 3. The brackets — the axes — are what change.

Watch out

`shape` is a tuple, not a number. `T.shape[0]` is the size of axis 0 (= 2, the branches), `len(T.shape)` is `ndim` (= 3), and `T.numel()` is the product (= 24). Mixing these up ("I have 3 things") is the first debugging trap; print all of `T.shape`, `T.ndim`, `T.numel()` when a shape error appears.

3 Step 2 — From shape to strides: the row-major layout

The 24 numbers live in memory as one flat line, in *row-major* (C-contiguous) order: the *last* axis varies fastest. Walking T in storage order means holding i, j fixed and sweeping $k = 0, 1, 2, 3$, then bumping j , and only after a full 3×4 table bumping the branch i :

$$\underbrace{0, 1, 2, 3}_{i=0, j=0}, \underbrace{4, 5, 6, 7}_{i=0, j=1}, \underbrace{8, 9, 10, 11}_{i=0, j=2} \mid \underbrace{12, \dots, 15}_{i=1, j=0}, \dots, \underbrace{20, 21, 22, 23}_{i=1, j=2}.$$

A **stride** is how many elements you skip in this flat line to advance one step along a given axis. For a contiguous tensor the stride of an axis is the product of the sizes of *all axes after it*:

$$\boxed{s_a = \prod_{b>a} d_b} \quad \Rightarrow \quad \begin{array}{ll} s_2 = 1 & \text{(empty product — last axis is packed)} \\ s_1 = d_2 = 4 & \text{(skip one full row of 4 items to reach the next day)} \\ s_0 = d_1 d_2 = 12 & \text{(skip one full } 3 \times 4 \text{ table to reach the next branch)} \end{array}$$

So `strides` = $(s_0, s_1, s_2) = (12, 4, 1)$, in *elements*.

Intuition

Read the stride vector right-to-left as a set of "carry" sizes, like an odometer with mismatched wheels. The item wheel (k) clicks every 1 slot. After 4 item clicks the day wheel (j) turns once — so its stride is 4. After 3 day turns (= 12 slots) the branch wheel (i) turns once — stride 12. The strides are exactly the place-values of this mixed-radix number system.

Mini-example — this operation in isolation

The rule $s_a = \prod_{b>a} d_b$ on a plain 3×4 matrix (shape $(3, 4)$): the last axis is packed so $s_1 = 1$; advancing one row jumps a whole line of 4, so $s_0 = 4$. Strides = $(4, 1)$. A $2 \times 3 \times 4$ tensor just prepends one more place-value, $s_0 = 3 \times 4 = 12$. Add an axis on the left $\Rightarrow$ multiply in its successors' sizes; the existing strides are untouched.

4 Step 3 — The addressing equation: index $\rightarrow$ flat offset

Strides exist to answer one question: given a logical index (i, j, k) , *where in the flat line is that element?* The answer is the dot product of the index with the stride vector:

$$\text{offset}(i, j, k) = s_0 i + s_1 j + s_2 k = 12 i + 4 j + 1 k$$

This is why we chose `arange(24)`: the stored value equals its own offset, so every line below is self-checking.

Worked evaluations.

$$\begin{aligned} \text{offset}(0, 0, 0) &= 12 \cdot 0 + 4 \cdot 0 + 0 = 0 & T = 0 \checkmark \\ \text{offset}(0, 1, 0) &= 12 \cdot 0 + 4 \cdot 1 + 0 = 4 & T = 4 \checkmark \\ \text{offset}(1, 0, 0) &= 12 \cdot 1 + 4 \cdot 0 + 0 = 12 & T = 12 \checkmark \\ \text{offset}(0, 2, 1) &= 12 \cdot 0 + 4 \cdot 2 + 1 = 9 & T = 9 \checkmark \\ \text{offset}(1, 2, 3) &= 12 \cdot 1 + 4 \cdot 2 + 3 = 23 & T = 23 \checkmark \end{aligned}$$

The last is the final corner of the box: the largest index $(d_0 - 1, d_1 - 1, d_2 - 1)$ maps to the largest offset `numel - 1 = 23`, as it must for a contiguous tensor.

Mini-example — this operation in isolation

Indexing in reverse: which element sits at flat offset 17? Peel off place-values largest-first. $17 \div 12 = 1 \text{ rem } 5$, so $i = 1$. $5 \div 4 = 1 \text{ rem } 1$, so $j = 1$. The remainder 1 is k . Thus $\text{offset } 17 \leftrightarrow (1, 1, 1)$, and indeed $T[1, 1, 1] = 17$. Forward (*index* $\rightarrow$ *offset*) is a dot product; backward (*offset* $\rightarrow$ *index*) is repeated division — the same arithmetic **reshape** uses to re-index the identical bytes.

Watch out

A row of $T[1]$ such as $[12, 13, 14, 15]$ is *not* a fresh array — it is a *view*: same storage, offset shifted to 12, reading 4 contiguous elements. That is why slicing a tensor is essentially free and why writing into a slice mutates the parent. The bytes never moved; only the (offset, shape, stride) recipe changed.

5 Step 4 — dtype: from elements to bytes

Strides and offsets are counted in *elements*. To reach actual memory you multiply by the **item size** — the number of bytes one element occupies, fixed by the **dtype**:

$$\text{bytes used} = \text{numel} \times \text{itemsize}, \quad \text{byte address}(i, j, k) = \text{offset}(i, j, k) \times \text{itemsize}.$$

For our `float32` tensor (`itemsize = 4`):

$$\text{bytes} = 24 \times 4 = 96 \text{ bytes}, \quad \text{element } (1, 2, 3) \text{ starts at byte } 23 \times 4 = 92.$$

The same 24 numbers cost different amounts under different dtypes — this table is the entire reason mixed precision saves memory:

dtype	itemsize (bytes)	total for 24 elems	relative	typical use
float64	8	192	$2.0\times$	scientific / reference precision
float32	4	96	$1.0\times$	default for weights & activations
float16	2	48	$0.5\times$	mixed-precision training
int64	8	192	$2.0\times$	class labels, indices
int8	1	24	$0.25\times$	quantised inference
bool	1	24	$0.25\times$	masks

Casting changes the bytes, and can change the value. Take $x = [1.7, 2.9, 3.1]$ in `float32`. Casting to an integer type *truncates toward zero* (it does not round):

$$[1.7, 2.9, 3.1] \xrightarrow{\text{.to(int32)}} [1, 2, 3].$$

Casting to `float16` keeps the value but snaps it to the nearest representable half-precision number:

$$[1.7, 2.9, 3.1] \xrightarrow{\text{.half()}} [1.70020, 2.90039, 3.09961].$$

Intuition

`float16` has only 10 fraction bits, so it can store about $2^{10} = 1024$ evenly spaced values between consecutive powers of two. Near 3.1 that spacing is $2^1/2^{10} = 2/1024 \approx 0.00195$, so 3.1 lands on the nearest grid point 3.09961 — off by $\sim 4 \times 10^{-4}$. Halving memory is not free; you pay in precision. That trade is exactly why training runs the *forward* pass in `float16` but keeps a `float32` master copy of the weights.

Watch out

Integer cast *truncates*, it does not round: `(0.9).to(int)` is 0, not 1. And an `int64` label tensor accidentally created as `float32` (e.g. `torch.tensor([1.0, 2.0])`) will be rejected by `CrossEntropyLoss`, which demands `int64` targets. Check `dtype` the moment a loss or index op complains.

6 Step 5 — device: same math, different memory

The **device** says *which* physical memory holds the flat line: `cpu` (system RAM) or `cuda:0` (GPU VRAM). Crucially, none of the arithmetic above changes — shape, strides, the offset equation, item size are all identical on GPU. Moving `.to("cuda")` copies the same 96 bytes into a different chip and hands back a tensor with the same recipe.

When this is the right choice

Keep data on the `cpu` for loading, small bookkeeping, and anything that must touch Python or disk. Move to `cuda` for the dense linear algebra of training and inference, where thousands of arithmetic units apply the *same* offset math to thousands of elements at once. The win is parallelism over the identical layout — not a different layout.

Watch out

Every operand of an op must sit on the *same* device. Adding a `cpu` tensor to a `cuda` tensor raises `RuntimeError: Expected all tensors to be on the same device`. This is the single most common beginner crash; the fix is one `.to(device)` on the stray operand, not a redesign.

7 Step 6 — Why reshape is free and transpose is not

Because the data is just a flat line plus a recipe, any new shape whose product is still 24 and whose intended walk order matches the existing storage order needs *no copy* — only a new (shape, stride) pair.

Reshape $(2, 3, 4) \rightarrow (6, 4)$. The flat line $0, 1, \dots, 23$ is unchanged; we merely re-read it as 6 rows of 4. The new strides follow the same rule $s_a = \prod_{b>a} d_b$: $s_1 = 1$, $s_0 = 4$, i.e. strides = $(4, 1)$. Free.

Transpose: swap axes 1 and 2, shape $(2, 3, 4) \rightarrow (2, 4, 3)$. A transpose refuses to move data; it just *swaps the corresponding strides*:

$$\text{shape } (2, 3, 4), \text{ strides } (12, 4, 1) \xrightarrow{\text{swap axes 1,2}} \text{shape } (2, 4, 3), \text{ strides } (12, 1, 4).$$

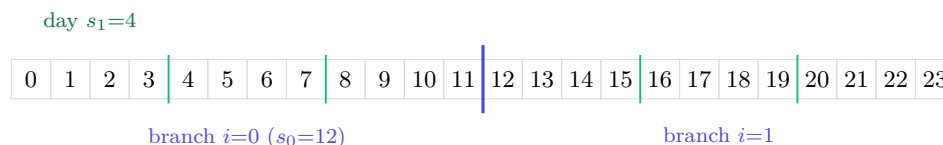
Now the last axis has stride 4, not 1 — the fast axis is no longer packed, so the tensor is **non-contiguous**. Reading it in logical order jumps around the flat line. Operations that assume a packed last axis then force a real copy via `.contiguous()`, which rewrites the bytes so strides become $(12, 3, 1)$ again.

Intuition

Contiguity is simply the question "does the stride vector still equal $\prod_{b>a} d_b$ for the current shape?" **reshape/view** preserve that relationship, so they are pointer tricks. **transpose/permute** break it by reordering strides without reordering bytes — cheap to create, but the next op that needs packed memory pays the copy. You now have the whole mental model the later "Reshape, View, Permute" and "Contiguous Tensors" topics formalise.

8 The layout, drawn

The flat line of 24 elements, with the axis boundaries the strides impose:



Thick indigo line = a branch step (stride 12); thin green lines = day steps (stride 4); each unit cell = an item step (stride 1). Element (i, j, k) sits at cell $12i + 4j + k$ — the addressing equation of Step 3, read straight off the strip.

9 Anatomy of a tensor, at a glance

#	Property	For our T	Computed as
1	<code>shape</code>	$(2, 3, 4)$	the axis lengths you declare
2	<code>ndim</code>	3	$\text{len}(\text{shape})$
3	<code>numel</code>	24	$\prod_a d_a$
4	<code>stride</code>	$(12, 4, 1)$	$s_a = \prod_{b>a} d_b$ (if contiguous)
5	offset of (i, j, k)	$12i + 4j + k$	$\sum_a s_a \text{idx}_a$
6	<code>dtype</code>	<code>float32</code>	chooses <code>itemsizes</code> = 4 B
7	<code>memory</code>	96 B	$\text{numel} \times \text{itemsizes}$
8	<code>device</code>	<code>cpu</code>	which chip holds the bytes
9	<code>is_contiguous</code>	<code>True</code>	$\text{strides} == \prod_{b>a} d_b?$

These nine numbers *are* the tensor. Print them first whenever something breaks; every shape, dtype, or device error in the rest of the course is one of these rows disagreeing with what an operation expected.

10 Equation cheat-sheet

Rank / <code>ndim</code>	$n = \text{len}(\text{shape})$
Element count	$\text{numel} = \prod_{a=0}^{n-1} d_a$
Contiguous stride	$s_a = \prod_{b>a} d_b$ (last axis: $s_{n-1} = 1$)
Index $\rightarrow$ offset	$\text{offset} = \sum_a s_a \text{idx}_a$
Byte address	$\text{offset} \times \text{itemsizes}$
Memory footprint	$\text{numel} \times \text{itemsizes}$
Transpose ($a \leftrightarrow b$)	swap $d_a \leftrightarrow d_b$ and $s_a \leftrightarrow s_b$ (no copy)
Contiguity test	$s_a \stackrel{?}{=} \prod_{b>a} d_b$ for all a

Appendix A Reference implementation

Every number above is reproducible from the following program (PyTorch; NumPy gives identical layout). Change the shape or dtype to explore.

```
import torch
T = torch.arange(24, dtype=torch.float32).reshape(2, 3, 4)

# --- the nine properties ---
print(T.shape)          # torch.Size([2, 3, 4])
print(T.ndim)           # 3
print(T.numel())        # 24
print(T.stride())        # (12, 4, 1)
print(T.dtype)           # torch.float32
print(T.element_size()) # 4 (bytes per element)
print(T.numel()*T.element_size()) # 96 (total bytes)
print(T.device)          # cpu
print(T.is_contiguous()) # True

# --- addressing equation: index -> flat offset ---
s = T.stride()
for idx in [(0,0,0),(0,1,0),(1,0,0),(0,2,1),(1,2,3)]:
    off = sum(si*ix for si, ix in zip(s, idx))
    assert T[idx] == T.flatten()[off] == off

# --- dtype & casting ---
x = torch.tensor([1.7, 2.9, 3.1])
print(x.to(torch.int32)) # tensor([1, 2, 3]) (truncation)
print(x.half())          # tensor([1.7002, 2.9004, 3.0996], dtype=float16)

# --- reshape is free; transpose breaks contiguity ---
print(T.reshape(6,4).stride()) # (4, 1)
Tt = T.transpose(1,2)          # shape (2,4,3)
print(Tt.stride(), Tt.is_contiguous()) # (12,1,4) False
print(Tt.contiguous().stride()) # (12,3,1)
```

— end of document —